Construction Heuristics for the Aircraft Landing Problem

TC-RBI (ramp_rbi.py) and ETRI (etri.py): rationale, mathematics, rules, pseudocode and complexity

October 2026 — code: const_heuristics/alp_core.py, etri.py, ramp_rbi.py

Contents

0. About this document
1. Problem and shared notation
    1.1 Data
    1.2 Model
    1.3 Two-stage decomposition and the Stage-2 LP
    1.4 Instance statistics used by the heuristics
PART I — TC-RBI (`ramp_rbi.py`)
2. TC-RBI: Target-Conflict Regret-Based Insertion
    2.1 Design principles
    2.2 Rationale
    2.3 Conflict queue
    2.4 Regret by queue resolution
    2.5 Selection (the only randomness) and commit
    2.6 Parameters
    2.7 Pseudocode
    2.8 Deterministic rules (reproduction recipe)
    2.9 Worked example: airland1, deterministic
    2.10 Design variants tested
    2.11 Complexity derivation
    2.12 Properties and limitations
PART II — ETRI (`etri.py`)
3. ETRI: Exact-Timing Randomised Insertion
    3.1 Rationale and design history
    3.2 Exact timing of one runway sequence: PAVA
    3.3 Insertion slots
    3.4 Pilot score
    3.5 Selection rule (the only randomness)
    3.6 Commit
    3.7 Pool generation and uniqueness
    3.8 Parameters
    3.9 Pseudocode
    3.10 Deterministic rules (reproduction recipe)
    3.11 Worked example: airland1, deterministic
    3.12 Complexity derivation
    3.13 Properties and limitations
4. Side-by-side comparison
5. Validation procedure
6. Results on airland1–13
    6.1 Deterministic construction
    6.2 Pools of 8 distinct schedules
    6.3 Randomness, diversity and local optimality
    6.4 Effect on SA
7. Using the heuristics as SA seeds
8. Reproducibility notes
9. References
Appendix A — airland1 data
Appendix B — full traces

B.1 TC-RBI, airland1, m = 2, deterministic
B.2 ETRI, airland1, m = 2, deterministic

0. About this document

This document specifies the two construction heuristics in `const_heuristics/`:

File	Content	Role
`ramp_rbi.py`	**TC-RBI** — Target-Conflict Regret-Based Insertion	Regret insertion driven by the current conflict queue; deterministic or randomised; seeds simulated annealing (SA)
`etri.py`	**ETRI** — Exact-Timing Randomised Insertion	Pilot-scored insertion with a fixed look-ahead; deterministic or randomised; seeds SA
`alp_core.py`	shared kernels	exact timing (PAVA) with LP fallback, insertion slots, schedule pools, benchmark driver

For each heuristic it gives the design rationale, every equation the code evaluates, pseudocode, a numbered rule list that reproduces the deterministic output, a worked trace on a small instance, a complexity derivation with a measured-scaling figure, and known limitations. Both are construction heuristics: there is no improvement or tuning phase. Section 6 compares them on the OR-Library benchmark, including how their randomised modes generate diverse near-optimal schedules; Section 7 shows how to pass their output to the SA solvers in this repository.

Indexing convention. Everything follows the code: aircraft are numbered $0, ..., n-1$ in data-file order, runways $\rho = 0, ..., m-1$ (printed as runway 1..m in traces), and positions in a runway sequence are $p = 0, ..., L$ where L is the current runway length. "Insert j at position p" means j becomes element p and the former elements $p, ..., L-1$ shift right by one.

---

1. Problem and shared notation

1.1 Data

Symbol	Meaning	Code
n, m	number of aircraft, number of identical runways	`n`, `m`
r_j	earliest landing time	`r`
δ_j	target landing time ($r_j \le \delta_j \le d_j$)	`delta` / `dl`
d_j	latest landing time	`d`
g_j, h_j	penalty per unit of earliness / tardiness	`g`, `h`
s_{ij}	minimum separation when i lands before j **on the same runway** ($s_{jj} = 0$)	`s`

There is no inter-runway separation: aircraft on different runways do not constrain each other.

1.2 Model

A schedule is a set of runway sequences $\Pi = (\pi_0, ..., \pi_{m-1})$ that together contain every aircraft once, plus landing times x_j:

$$\min \sum_j g_j \left(\delta_j - x_j\right)^+ + h_j \left(x_j - \delta_j\right)^+$$

$$\text{s.t.} \quad r_j \le x_j \le d_j \ \ \forall j, \qquad x_{\pi_\rho[b]} \ge x_{\pi_\rho[a]} + s_{\pi_\rho[a]\,\pi_\rho[b]} \ \ \forall \rho,\ \forall\, a < b.$$

Note the separation constraint covers **all** ordered pairs on a runway, not only consecutive ones. The OR-Library matrices violate the triangle inequality ($s_{ik} > s_{ij} + s_{jk}$ for some triples; airland8 has 9 802 such triples), so consecutive separations alone do not imply the others.

1.3 Two-stage decomposition and the Stage-2 LP

Both heuristics only decide Π (assignment and order). Once Π is fixed, the optimal times come from a linear program that separates by runway, $F(\Pi) = \sum_\rho f(\pi_\rho)$, with

$$f(\pi) = \min_{x,\,E,\,T} \sum_k g_{\pi_k} E_k + h_{\pi_k} T_k \quad \text{s.t.} \quad \begin{array}{ll} x_k + E_k \ge \delta_{\pi_k}, & x_k - T_k \le \delta_{\pi_k}, \\ x_b - x_a \ge s_{\pi_a \pi_b} \ \ (a < b), & r_{\pi_k} \le x_k \le d_{\pi_k},\ E, T \ge 0. \end{array}$$

$f(\pi) = \infty$ when the LP is infeasible. The reported objective of either heuristic is $F(\Pi)$ computed by this LP (`alp_core.Problem.exact_cost`), and every schedule is checked independently with `alp_core.audit` (§5).

1.4 Instance statistics used by the heuristics

Symbol	Definition	Used by
p_j	$\max(g_j, h_j)$	both
$\bar{s}$	mean of the strictly positive off-diagonal s_{ij} (1.0 if none)	both
$s_{\max}$	$\max_{i,j} s_{ij}$	both (PAVA exactness check, LP row pruning)
c_{unit}	$\left(\frac{1}{n}\sum_j p_j\right)\bar{s}$: the cost of delaying an average aircraft by one average separation	both (temperature unit)

PART I — TC-RBI (`ramp_rbi.py`)

2. TC-RBI: Target-Conflict Regret-Based Insertion

2.1 Design principles

- **Every quantity is a cost in the objective's units.** Insertions are priced by the exact increase of the runway's Stage-2 cost (PAVA, §3.2). No cost terms are weighted against

each other, so there is nothing to tune.
- **Only contested aircraft are compared.** The candidate set is the conflict queue (§2.3): the aircraft whose order is actually in question now.
- **Selection follows the regret principle exactly** (§2.2, §2.4).
- **Feasibility is exact.** Every commit is priced with the exact Stage-2 optimum. An aircraft that a competitor would make infeasible is resolved first, through $D = \infty$.
- **Construction only.** There is no improvement or tuning phase. The only randomness is in which aircraft is inserted next (§2.5).

On the 16 hardest jobs the deterministic construction is about 1% from the best known values for one runway and about 0.1% for several (§2.10).

2.2 Rationale

Regret insertion (Potvin & Rousseau 1993) delays the aircraft that can afford to wait and inserts first the one that would lose most by waiting. The design builds on two observations.

Regret has an exact pairwise meaning. Let $c(j)$ be the cheapest insertion cost of j in the current partial schedule, and $c(j \mid k)$ its cheapest cost after k has been placed at its own cheapest slot. The regret of j when k goes first is $r(j \mid k) = c(j \mid k) - c(j)$. Of two aircraft, inserting j first costs $c(j) + c(k \mid j)$ and inserting k first costs $c(k) + c(j \mid k)$, so

$$j \text{ first} \Leftrightarrow c(j) + c(k \mid j) \leq c(k) + c(j \mid k) \Leftrightarrow r(k \mid j) \leq r(j \mid k).$$

The aircraft whose regret is larger goes first, with no weight involved.

Only aircraft in conflict need regret. If two aircraft can both land on target, their order does not matter and regret is zero. The *target conflict* of TC-RBI's name identifies which aircraft compete. Measuring it from target times alone misses the main source of competition: congestion. Once a runway is busy, every aircraft queued behind it competes for the next slot, even if their targets are far apart. So TC-RBI identifies competitors from the current congestion state (§2.3).

2.3 Conflict queue

Keep the pending aircraft in target order, $\Pi^{pend} = (k_1, k_2, \ldots)$, built with a stable sort on δ.

For every non-empty runway ρ, let $last(\rho)$ be its last aircraft and $t(\rho)$ that aircraft's landing time in the PAVA optimum of the current runway sequence.

Projection. Process $k_1, k_2, \ldots$ first-come-first-served:

$$e_\rho(k) = \max\left(\delta_k,\ t(\rho) + s_{last(\rho)\,k}\right) \quad (e_\rho(k) = \delta_k \text{ if } \rho \text{ is empty}), \qquad \rho^*(k) = \operatorname*{argmin}_\rho e_\rho(k), \quad \tau_k = e_{\rho^*}(k),$$

with ties going to the lowest runway index. Then set $last(\rho^*) = k$ and $t(\rho^*) = \tau_k$.

Queue. $Q = (k_1, \ldots, k_q)$. Aircraft k_t is appended if $t \leq \kappa_{\min}$, or if it is

- **delayed**: $\tau_{k_t} > \delta_{k_t} + 10^{-9}$, or
- **in target conflict** with its predecessor: $\delta_{k_t} - \delta_{k_{t-1}} < s_{k_{t-1}k_t}$.

The first aircraft that meets none of these conditions ends the queue, and so does reaching the cap ($q \leq$ `queue_cap` $= 14$).

Minimum length $\kappa_{\min} = \min(m+1, |\Pi^{pend}|)$. With m runways the next $m+1$ aircraft must put two on the same runway (pigeonhole), so they always compete. Without this floor the projection seldom shows delays on several runways, the queue collapses to one aircraft and the choice becomes myopic. The multi-runway gap was 0.44% without the floor and 0.11% with it.

The cap is a computational bound. Measured queues average 5–7 aircraft for one runway (8–29% of steps reach the cap) and about 4 for two runways (never at the cap).

2.4 Regret by queue resolution

For a member $j \in Q$ and a slot $\sigma = (\rho, p)$ in its window, the **resolution cost** is

$$D(j,\sigma) = \sum_{\rho'} F'_{\rho'} \quad \text{after inserting } j \text{ at } \sigma \text{ and then every } b \in Q\backslash\{j\}, \text{ in queue order, at its cheapest slot.}$$

The slots, the window and the "cheapest slot" are exactly those of §3.3: $W = w_b + w_f + 1 = 8$ positions per runway around the target-order position, priced with PAVA (a lower bound when not exact), with ties broken by lower runway and then lower position. $D = \infty$ if some member is left with no feasible slot. The **score** of j is $D^*(j) = \min_\sigma D(j,\sigma)$.

Relation to regret. For a queue of two members $\{j,k\}$, where j's best D-slot is its cheapest slot,

$$D^*(j) = F + c(j) + c(k \mid j), \qquad \text{so} \qquad D^*(j) \le D^*(k) \Leftrightarrow r(k \mid j) \le r(j \mid k),$$

which is exactly the pairwise regret rule of §2.2. In general $D^*(j) \le F + c(j) + c(k \mid j)$: the score also lets j take the slot that minimises its own cost plus the regret it inflicts on the rest of the queue.

For larger queues the other members respond in sequence (in target order) rather than independently. Independent responses would let them all claim the same freed slot; that variant was tested and came out much worse (§2.10).

2.5 Selection (the only randomness) and commit

- **Deterministic** (`rng=None`): choose the member with the smallest D^*; ties go to the earliest in the queue (target order).
- **Randomised** (`rng` given): draw member j with probability $\propto \exp\left(-(D^*(j) - \min_Q D^*)/(\tau c_{unit}) \right)$, with $\tau = 0.3$ and c_{unit} as in §1.4.

Commit. The member's scored slots are tried in increasing D; exact ties keep slot-option order. The first slot whose new runway sequence has a finite **exact** cost (PAVA, or the pruned pairwise LP when flagged) is committed. If all of a member's slots fail, the next member in score order is tried.

Fallback. If no member has a finite D in the window, the scores are recomputed with the window widened to every position. If that also fails, construction stops with `(None, inf)`; this never happened on airland1–13.

`tc_rbi_pool` builds pools with the same canonical-form deduplication and temperature escalation as ETRI (`alp_core.build_pool`, §3.7).

2.6 Parameters

Parameter	Default	Meaning
`queue_cap`	14	maximum queue length (computational bound)
`w_back`, `w_fwd`	6, 1	slot window around the target-order position
`temperature` τ	0.3	selection temperature in units of c_{unit} (randomised mode only)

Measured sensitivity (deterministic, mean gap; single runway = airland8–13, multi-runway = airland9–13 with m = 2–4):

Setting	Single runway	Multi-runway
cap 6, $\kappa_{\min} = 1$	4.97%	0.44%
cap 10, $\kappa_{\min} = 1$	1.75%	0.44%
cap 14, $\kappa_{\min} = 1$	1.22%	0.44%
cap 20, $\kappa_{\min} = 1$	1.38%	0.44%
cap 30, $\kappa_{\min} = 1$	1.39%	0.44%
cap 14, $\kappa_{\min} = m + 1$ (default)	**1.09%**	**0.11%**
cap 14, window (10, 2), $\kappa_{\min} = 1$	1.12%	0.44%

The gain levels off at a cap of 14. A wider window helps marginally but doubles the time.

2.7 Pseudocode

```
TC-RBI(inst, m, rng=None, prm):
    Π^pend ← stable argsort of δ;   π_ρ ← (), F_ρ ← 0 for every runway ρ
    while Π^pend ≠ ∅:
        Q ← CONFLICT-QUEUE(min(m+1, |Π^pend|), prm.queue_cap)        # §2.3
        rows ← {(j, σ, D(j,σ)) : j ∈ Q, σ ∈ window slots of j, D < ∞}  # §2.4
        if rows = ∅: rows ← same with the window = all positions
        if rows = ∅: return FAIL
        D*(j) ← min over j's rows
        if rng = None: order ← members by (D*, queue position)
        else:          j0 ~ softmax(−(D* − min D*)/(τ·c_unit))
                       order ← [j0] + remaining members by D*
        for j in order:
            for σ in j's slots by (D, slot order):
                c ← EXACT-COST(insert(π_ρ(σ), j, p(σ)))
                if c < ∞: commit j at σ; F_ρ ← c; remove j from Π^pend; next iteration
        return FAIL
    return Π, Σ_ρ F_ρ

CONFLICT-QUEUE(kmin, cap):
    (last(ρ), t(ρ)) ← last aircraft of each runway and its PAVA landing time
    Q ← []
    for k in Π^pend:
        ρ* ← argmin_ρ e_ρ(k);   τ ← e_ρ*(k)
        delayed  ← τ > δ_k
        conflict ← Q ≠ [] and δ_k − δ_last(Q) < s_{last(Q), k}
        if |Q| ≥ kmin and not (delayed or conflict): break
        append k to Q;   last(ρ*) ← k;   t(ρ*) ← τ
        if |Q| = cap: break
    return Q

D(j, σ):                                    # on a copy of the state
    insert j at σ
    for b in Q \ {j}, in order:
        O ← finite slots of b;   if O = ∅: return ∞
```

```
        insert b at O[0]
    return total cost of the copy
```

2.8 Deterministic rules (reproduction recipe)

Following these rules with `rng=None` and default parameters reproduces `tc_rbi(inst, m)` exactly. Tolerances: 10^{-9} for windows and delay, 10^{-12} for PAVA merges, 10^{-7} for pairwise checks. Cost ties are exact floating-point equalities.

- **T1.** Sort aircraft by δ with a stable sort (equal targets by index): this is the pending list. Start with m empty runways of cost 0.
- **T2.** Compute each runway's last aircraft and its PAVA landing time. Build the queue Q by the projection and stopping rule of §2.3, with $\kappa_{\min} = \min(m+1, \#\text{pending})$ and cap 14.
- **T3.** For each member in queue order, list its slots exactly as ETRI rule **E3** does (window $[q-6, q+1]$, PAVA values, stable sort by increase over the enumeration order runway → position). For each slot, compute D (§2.4): insert the member, then every other member in queue order at the first entry of its own slot list on the copy. Keep finite values.
- **T4.** If no member has a finite D, redo T3 with the window $[0, L]$ on every runway. If still nothing is finite, stop with failure.
- **T5.** A member's score is its smallest D. Its slots are ordered by D with a stable sort, so equal D keeps the T3 order.
- **T6.** Choose the member with the smallest score; ties go to the earliest in the queue.
- **T7.** Try that member's slots in order. Commit the first whose new runway has a finite exact cost (PAVA, or the pruned LP when flagged): insert the aircraft, set the runway cost, and remove it from the pending list. If no slot works, try the next member in score order (ties by queue order).
- **T8.** Repeat T2–T7 until nothing is pending. The output is the runway sequences and $\sum_\rho F_\rho$, which equals the Stage-2 LP objective.

Randomised variant: replace T6 with the softmax draw of §2.5.

2.9 Worked example: airland1, deterministic

m = 1 (generated from the code and asserted equal to `tc_rbi()`; data in Appendix A):

Step	Queue Q	D^* of every member	Picked	Pos	Sequence after	Cost
1	2, 3	0, 0	2	0	[2]	0
2	3, 4	0, 0	3	1	[2,3]	0
3	**4, 5, 6, 7, 8, 0, 9**	700 ×7	4	2	[2,3,4]	0
4	5, 6, 7, 8, 0, 9	700 ×6	5	3	[2,3,4,5]	0
5	6, 7, 8, 0, 9	700 ×5	6	4	[2,3,4,5,6]	150
6	7, 8, 0, 9	700 ×4	7	5	[2,...,6,7]	360
7	8, 0, 9	700 ×3	8	6	[2,...,7,8]	480
8	0, 9	700 ×2	0	7	[2,...,8,0]	620
9	9, 1	700 ×2	9	8	[2,...,0,9]	700
10	1	700	1	9	[2,3,4,5,6,7,8,0,9,1]	**700**

Reading the table:

- **Steps 1–2.** The queue has the minimum two members, there is no congestion, and both D^* values are 0.
- **Step 3.** Aircraft 2 and 3 occupy the runway, and aircraft 4–9 and 0 have targets 123–180 with separations of 8–15. In the first-come-first-served projection each of them is either delayed or in target conflict with its predecessor, so the queue grows to the whole rush (7 aircraft).
- **Steps 3–10.** The resolution cost of every member is already 700, the optimum, because resolving the whole rush inside D foresees the final cost. Ties then go to queue order. ETRI reaches the same cost by a different sequence (§3.11).

m = 2: final schedule [2,3,4,5,7,8,9,1] · [6,0] with cost **90.00** (optimal). Appendix B.1 has the full trace.

2.10 Design variants tested

All variants are deterministic and use exact costs. The table shows the mean gap to the best known values, using the improved single-runway references 12292.20 for airland10 and 37165.39 for airland13.

Variant	Single runway (6 jobs)	Multi-runway (10 jobs)
Target order, cheapest slot (no regret)	15.70%	15.59%
Target-conflict cluster + sum of pairwise regrets, cheapest slot	9.88%	1.58%
… + slot chosen by *independent* best responses of the cluster	11.32%	11.08%
Target-conflict cluster + queue resolution	8.06%	0.44%
Congestion queue + sum of pairwise regrets, cheapest slot	8.01%	1.58%
Congestion queue + queue resolution ($\kappa_{\min} = 1$)	1.22%	0.44%
Congestion queue ($\kappa_{\min} = m + 1$) + queue resolution (final)	**1.09%**	**0.11%**

Two lessons:

- **The congestion state matters more than target times** for deciding who competes.
- **Resolving the queue in sequence is far better than summing pairwise regrets.** The pairwise sum treats each comparison in isolation and ignores how the responses interact.

2.11 Complexity derivation

Notation: k is the number of aircraft scheduled, $\bar{L} = k/m$, $W = w_b + w_f + 1$, $q = |Q| \leq$ `queue_cap`, and $C(L)$ the PAVA cost (§3.12: $O(L)$ typical, $O(L^2)$ worst).

- **Queue** (§2.3): one PAVA call per runway plus a projection that stops after at most $q + 1$ aircraft: $O\left(m\, C(\bar{L}) + qm\right)$.
- **Slot list of one aircraft:** $O\left(mW C(\bar{L})\right)$, as for ETRI.

- **One resolution** $D(j,\sigma)$**:** a state copy of $O(mn)$ plus $q-1$ slot lists and commits: $O\left(\, mn + q\,mW\,C(\bar{L}) \,\right)$.
- **All scores** (up to $q \cdot mW$ resolutions):

$$T_{scores}(k) = O\left(\, q\,mW\,(mn + q\,mW\,C(k/m)) \,\right) = O\left(\, q\,m^2Wn + q^2m^2W^2\,C(k/m) \,\right).$$

- **Commit:** $O(C(L))$, with a rare LP, plus $O(qmW\log(qmW))$ for sorting and $O(n)$ for list maintenance.

Whole construction, writing $\bar{q^2}$ and $\bar{q}$ for the means over steps:

- Typical, $C(L) = cL$: $\sum_{k<n}\left(\, \bar{q}\,m^2Wn + \bar{q^2}\,m^2W^2\,k/m \,\right) = \bar{q}\,m^2Wn^2 + \bar{q^2}\,mW^2\,\frac{n(n-1)}{2}$, so

$$T_{TC\text{-}RBI} = O\left(\, \bar{q^2}\,m\,W^2\,n^2 + \bar{q}\,m^2Wn^2 \,\right).$$

- Worst case, $C(L) = L^2$: $\sum_k \bar{q^2}\,m^2W^2(k/m)^2 = \bar{q^2}\,W^2\,\frac{(n-1)n(2n-1)}{6}$, so $O\left(\, \bar{q^2}\,W^2n^3 + \bar{q}\,m^2Wn^2 \,\right)$.

The bound $q \le 14$ keeps $\bar{q^2} \le 196$. Measured values of $\bar{q^2}$ are 42–73 for one runway and 19–23 for two. With $W = 8$, the typical constant $\bar{q^2}\,mW^2/2$ is about 1 300–2 300 for one runway, against about 520 for ETRI ($KTHW/2$), which matches TC-RBI being 2–4× slower.

Empirical check (one deterministic construction, best of 3 runs; Figure 1):

n	**100**	**150**	**200**	**250**	**500**
m = 1 (s)	0.185	0.923	0.718	1.064	7.381
m = 2 (s)	0.123	0.297	0.349	0.562	2.224

The time follows $\bar{q^2}\,n^2$, not n^2 alone. airland10 (n = 150) and airland13 (n = 500) are the most congested single-runway instances, with $\bar{q^2} = 73$ and 67, which is why their times stand out.

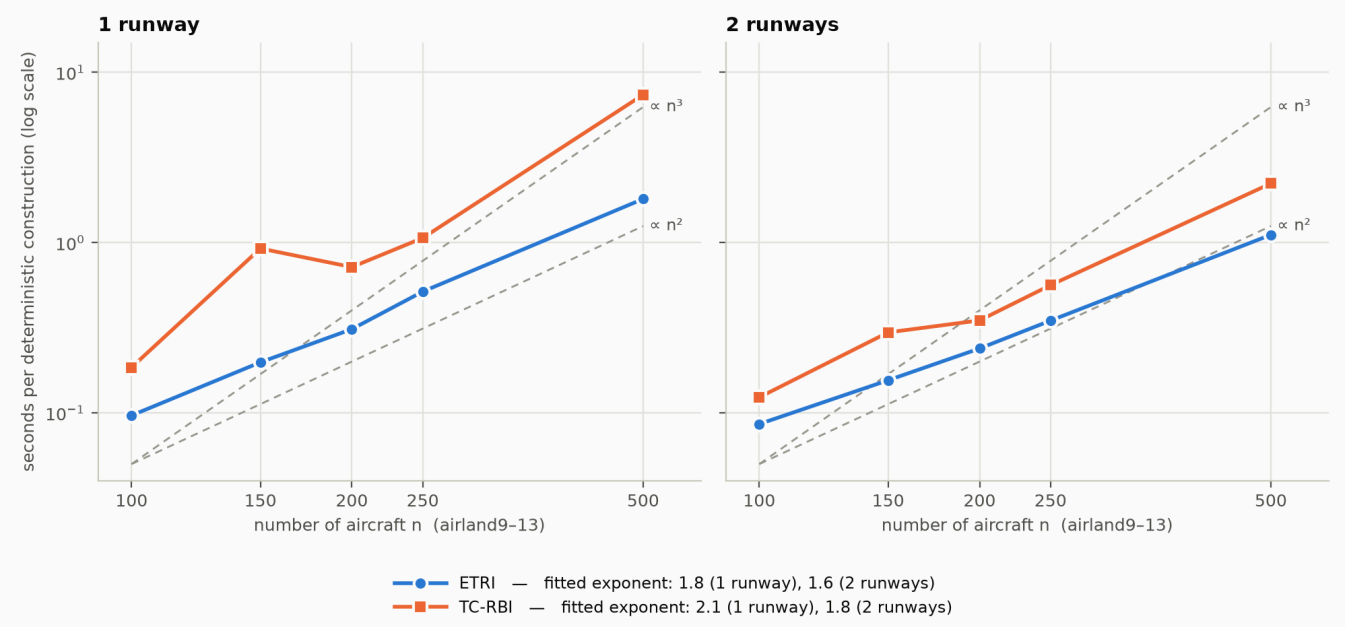


Figure 1 — Measured construction time against instance size (log–log), one deterministic construction per point on airland9–13. Dashed gray lines are reference slopes $\propto n^2$ and $\propto n^3$; a line parallel to a reference grows at that rate. Fitted exponents: ETRI 1.8 (one runway) and 1.6 (two runways); TC-RBI 2.1 and 1.8. TC-RBI's bumps at n = 150 and 500 are the congested instances airland10 and airland13, where the mean squared queue length $\bar{q^2}$ is largest (§2.11).

Memory: $O(n^2)$ for s, $O(mn)$ for the state and one live copy, and $O(qmW)$ score rows.

2.12 Properties and limitations

- **Feasible and exactly costed**, like ETRI: every committed runway is priced with the exact Stage-2 optimum, and every benchmark schedule passed the repository's pairwise audit.
- **Weight-free.** The parameters are a computational cap, the slot window and the randomness level; none trades one cost against another.
- **Resolution uses PAVA values without the exactness flag,** so on triangle-violating instances a score can be optimistic. The commit is always exact.
- **The queue cap truncates long rushes.** On congested single-runway instances 8–29% of steps reach the cap of 14.
- **Not guaranteed to be a local optimum** (§6.3); improvement is left to SA.
- **Relation to ETRI.** With exact costs the two heuristics are close relatives: both pick an aircraft by the cost of a look-ahead completion. They differ in what drives the look-ahead:
  - TC-RBI's candidate set and look-ahead are the same adaptive set, the conflict queue, and every window slot is scored.
  - ETRI uses a fixed candidate set ($K = 4$), horizon ($H = 8$) and slot count ($T = 4$).

  TC-RBI wins on single-runway quality; ETRI is 2–4× faster.

PART II — ETRI (`etri.py`)

3. ETRI: Exact-Timing Randomised Insertion

3.1 Rationale and design history

ETRI was designed to produce **initial solutions for SA**: high-quality, always feasible, and diverse when needed. Four observations drove the design.

1. **Exact timing is cheap if computed the right way.** For a fixed runway sequence the consecutive-separation timing problem is an isotonic regression, solved exactly by the pool-adjacent-violators algorithm (PAVA, §3.2). PAVA matched HiGHS on all 530 test sequences and is about 150× faster (36 µs vs 5.6 ms at $n = 500$). So every insertion can be priced with the real objective instead of a surrogate, and no weights are needed to combine cost terms.

2. **Near-optimal sequences stay close to target order.** In near-best single-runway sequences for airland9, 10 and 12, the mean displacement from target-time order is 0.4–0.7 positions (max 3–8). ETRI therefore processes aircraft in target order and only searches a small window of positions around each aircraft's target-order slot.

3. **Myopic insertion fails during congestion.** Inserting the next aircraft at its cheapest slot ignores the delay it pushes onto the aircraft behind it. Measured on the 16 hardest jobs (construction only, without the pilot):

Selection rule (deterministic)	Single-runway gap (6 jobs)	Multi-runway gap (10 jobs)
Target order + cheapest slot	15.7%	15.6%
Regret over runways (look-ahead 6)	15.7%	1.1%
Pilot on the cheapest slot only ($K = 4$, $H = 4$)	8.2%	0.66%
Pilot on the cheapest slot only ($K = 4$, $H = 12$)	6.8%	0.46%
Pilot over each candidate's 4 cheapest slots ($K = 4$, $H = 8$, $T = 4$, final)	**2.4%**	**0.03%**

The single-runway jobs are airland8–13 and the multi-runway jobs are airland9–13 with m = 2–4. Gaps are mean percentage gaps to BKS.

The *pilot method* (Duin & Voß 1999) scores each choice by tentatively completing the next few insertions greedily.

4. **Randomness belongs in the choice of aircraft.** The aircraft to insert is drawn from the feasible candidate set with a probability that falls off with its pilot score. Close calls get randomised, while clearly worse aircraft are almost never chosen, which gives diverse schedules without destroying quality.

Earlier prototypes also had a local search, order noise and an insertion RCL. These were removed: improvement is SA's job, and randomness was restricted to aircraft selection.

3.2 Exact timing of one runway sequence: PAVA

Transformation. Let $\pi = (\pi_0, ..., \pi_{L-1})$ and define the cumulative consecutive separation

$$S_0 = 0, \qquad S_k = S_{k-1} + s_{\pi_{k-1}\pi_k}, \qquad y_k = x_k - S_k.$$

The consecutive constraint $x_k - x_{k-1} \geq s_{\pi_{k-1}\pi_k}$ becomes $y_k \geq y_{k-1}$. The relaxed (consecutive-only) timing problem is therefore

$$(P_c): \quad \min_y \sum_{k=0}^{L-1} \phi_k(y_k) \quad \text{s.t. } y_0 \leq y_1 \leq ... \leq y_{L-1},$$

$$\phi_k(y) = g_{\pi_k}(t_k - y)^+ + h_{\pi_k}(y - t_k)^+ + \mathbb{I}\left[lo_k \leq y \leq hi_k\right], \quad t_k = \delta_{\pi_k} - S_k,\ lo_k = r_{\pi_k} - S_k,\ hi_k = d_{\pi_k} - S_k,$$

where $\mathbb{I}[\cdot]$ is 0 inside the interval and $+\infty$ outside. Each ϕ_k is convex and piecewise linear. This is isotonic regression with separable convex losses, which PAVA solves exactly (Best, Chakravarti & Ubhaya 2000; Ahuja & Orlin 2001).

PAVA. Maintain a stack of blocks of consecutive positions. Every block B holds one common value y_B, its bounds $LO_B = \max_{k \in B} lo_k$ and $HI_B = \min_{k \in B} hi_k$, and its breakpoints t_k sorted with their slopes (g, h). Push positions one at a time as singleton blocks. While the top two blocks violate monotonicity ($y_{\text{below}} > y_{\text{top}} + 10^{-12}$), merge them: merge the sorted breakpoint lists, intersect the bounds, and recompute y_B. If ever $LO_B > HI_B + 10^{-9}$, the sequence is infeasible.

Block minimiser. For $\Phi_B(y) = \sum_{k \in B} \phi_k(y)$ on $[LO_B, HI_B]$, the right derivative of the linear part is

$$\Phi_B'^{+}(y) = \sum_{k \in B:\, t_k \le y} h_{\pi_k} \;-\; \sum_{k \in B:\, t_k > y} g_{\pi_k}.$$

Sort the breakpoints $t_{(1)} \le \ldots \le t_{(b)}$. The derivative starts at $-G_B$ with $G_B = \sum_{k \in B} g_{\pi_k}$ and jumps by $g_{(i)} + h_{(i)}$ at $t_{(i)}$. The **smallest minimiser** is

$$y_B^* = t_{(i^*)}, \qquad i^* = \min\Big\{ i : -G_B + \sum_{l \le i} \big(g_{(l)} + h_{(l)} \big) \ge 0 \Big\},$$

with $y_B^* = -\infty$ if $G_B = 0$. Because Φ_B is convex, the constrained minimiser is the clipped value $y_B = \min(\max(y_B^*, LO_B), HI_B)$. A singleton block gets $y = \min(\max(t_k, lo_k), hi_k)$.

Recovering the solution. $x_k = y_{B(k)} + S_k$, and the cost is $\sum_k g_{\pi_k}(\delta_{\pi_k} - x_k)^+ + h_{\pi_k}(x_k - \delta_{\pi_k})^+$.

Exactness test (Lemma). The feasible set of the full pairwise LP (P_f) is contained in that of (P_c), and both have the same objective. Hence:

- $f_c(\pi) \le f(\pi)$, so PAVA always gives a lower bound;
- if the optimum x of (P_c) satisfies every non-consecutive separation, then x is feasible for (P_f) and therefore optimal, so $f(\pi) = f_c(\pi)$;
- if (P_c) is infeasible, so is (P_f).

Complete pairwise check in $O(\kappa L)$. For $a < k-1$, if $S_k - S_a \ge s_{\max}$, then $x_k - x_a \ge S_k - S_a \ge s_{\max} \ge s_{\pi_a \pi_k}$ is implied by the consecutive constraints. So it is enough to scan $a = k-2, k-3, \ldots$ while $S_k - S_a < s_{\max}$. With $s_{\min}$ the smallest positive separation, this takes at most $\kappa = \lceil s_{\max}/s_{\min} \rceil$ steps per position (κ = 5 for airland1 and κ = 2 for airland9–13). A violation, with tolerance 10^{-7}, flags the result as *not exact*. `cet()` returns `(cost, exact)`.

PAVA, the exactness check and the LP fallback below are implemented in `alp_core.py` (`cet`, `Problem.exact_cost`, `lp_rows`) and shared by both heuristics.

When not exact: full LP with row pruning. The pairwise LP (§1.3) is solved with HiGHS. It keeps every consecutive row, and a row (a, b) with $b > a + 1$ only if it is not implied:

- by the chain: $P_b - P_a \ge s_{\pi_a \pi_b}$, with P the consecutive prefix sums;
- or by the windows: $r_{\pi_b} - d_{\pi_a} \ge s_{\pi_a \pi_b}$.

The scan over b stops once $P_b - P_a \ge s_{\max}$.

Worked example (4 aircraft, sequence 1, 2, 3, 4). The data are

Aircraft	r	δ	d	g	h
1	0	10	40	2	3
2	0	12	40	2	1
3	0	13	40	1	2
4	20	30	60	1	1

with consecutive separations $s_{12} = 4$, $s_{23} = 3$, $s_{34} = 4$ and non-consecutive $s_{13} = 5$, $s_{14} = s_{24} = 5$ (so $s_{\max} = 5$). Then $S = (0, 4, 7, 11)$, $t = (10, 8, 6, 19)$, $lo = (0, -4, -7, 9)$ and $hi = (40, 36, 33, 49)$.

1. Push aircraft 1: $y = 10$.
2. Push aircraft 2: $y = 8 < 10$, so merge. Breakpoints $(8: g = 2, h = 1), (10: g = 2, h = 3)$. The slope goes $-4 \to -1$ (after 8) $\to 4$ (after 10), so $y^* = 10$. Bounds $[0, 36]$, so $y = 10$.

3. Push aircraft 3: $y = 6 < 10$, so merge. Breakpoints $(6; 1, 2), (8; 2, 1), (10; 2, 3)$. The slope goes $-5 \to -2 \to +1$, so $y^* = 8$. Bounds $[0, 33]$, so $y = 8$.
4. Push aircraft 4: $y = 19 \geq 8$, so no merge.

Result: $x = (8, 12, 15, 30)$ and cost $= 2 \cdot 2 + 0 + 2 \cdot 2 + 0 = 8$. The pairwise scan stops immediately ($S_2 - S_0 = 7 \geq s_{\max}$), so the result is exact. HiGHS confirms 8.

Triangle variant. With $s_{13} = 9 > s_{12} + s_{23} = 7$ (so $s_{\max} = 9$), PAVA still returns cost 8, but $x_3 - x_1 = 7 < 9$ flags it as not exact. The LP gives the true value **12** at $x = (6, 12, 15, 30)$; aircraft 1 lands 2 units earlier at cost 4.

3.3 Insertion slots

For runway ρ holding σ (length L), define the **target-order position**

$$q_\rho(j) = \min\{k : \delta_{\sigma_k} > \delta_j\} \quad (\text{strict}; \ = L \text{ if none}).$$

The slot window is $p \in [\max(0, q_\rho(j) - w_b),\ \min(L, q_\rho(j) + w_f)]$, with $w_b = 6$ and $w_f = 1$, so at most $W = w_b + w_f + 1 = 8$ positions per runway. For each slot,

$$\Delta(j, \rho, p) = f_c\left(\text{ins}(\sigma, j, p)\right) - F_\rho,$$

where f_c is the PAVA value (a lower bound if not exact) and F_ρ the current cost of runway ρ. The **option list** $O(j)$ contains the finite slots, enumerated with ρ ascending then p ascending, and stably sorted by Δ. Ties therefore go to the lower runway, then the lower position.

3.4 Pilot score

- Π^{pend}: the pending aircraft in target order. The initial order is `argsort(δ, stable)`, so equal targets go by index.
- Candidate set $\mathcal{C}$: the first K pending aircraft. Look-ahead list Λ: the first H pending aircraft, with $K \leq H$ (defaults $K = 4$, $H = 8$).

For candidate j and slot $(\rho, p) \in O(j)$, the **pilot run** works on a copy of the state:

1. insert j at (ρ, p), with runway cost set to f_c;
2. for each $k \in \Lambda \backslash \{j\}$ in list order, insert k at the first option of $O(k)$ under the current copy; if $O(k)$ is empty, the pilot value is ∞;
3. the pilot value is $\Phi(j, \rho, p) = \sum_{\rho'} F'_{\rho'}$, the total cost of the copy.

The pilot score of j is the best value over its T cheapest slots (default $T = 4$):

$$\text{score}(j) = \min_{(\rho,p) \in O(j)[0..T-1]} \Phi(j, \rho, p).$$

The slots that were scored are kept sorted by (Φ, ρ, p). The **feasible set** is $\mathcal{F} = \{j \in \mathcal{C} : \text{score}(j) < \infty\}$.

Why the pilot score is comparable across candidates: for every candidate, the pilot ends with the same aircraft set $\Lambda \cup \{j\} = \Lambda$ scheduled (since $j \in \Lambda$). The candidates differ only in insertion order and slots.

3.5 Selection rule (the only randomness)

- **Deterministic** (`rng=None`): pick $j^\star = \operatorname{argmin}_{j \in \mathcal{F}} \text{score}(j)$; ties go to the earlier candidate in target order.
- **Randomised** (`rng` given): draw $j^\star$ with

$$\Pr(j) = \frac{\exp\Big(-(\text{score}(j) - \min_{\mathcal{F}}\text{score})/(\tau\, c_{unit})\Big)}{\sum_{l \in \mathcal{F}} \exp\Big(-(\text{score}(l) - \min_{\mathcal{F}}\text{score})/(\tau\, c_{unit})\Big)}, \qquad c_{unit} = \bar{p}\,\bar{s}.$$

c_{unit} is the cost of delaying an average aircraft by one average separation, so τ is unit-free across instances. The odds of a candidate Δ worse than the best are $e^{-\Delta/(\tau c_{unit})}$. With $\tau = 0.3$, a candidate one c_{unit} worse is drawn with relative weight 0.036, and one $0.3\,c_{unit}$ worse with 0.37.

Two alternatives were also tested: a softmax whose temperature is scaled by the score spread of the current candidates, and a value-based RCL. The c_{unit}-scaled rule was kept for two reasons. Its temperature means the same thing at every step and on every instance. And on the single-runway jobs it gave a better pool best (1.95% vs 2.2%) with more unique schedules (81% vs 47%).

3.6 Commit

The drawn aircraft's scored slots are tried in (Φ, ρ, p) order. Each new runway sequence is priced **exactly**: PAVA when the result is exact, otherwise the pruned LP (§3.2), with memoisation. The first slot with a finite exact cost is committed. If every slot of that aircraft is LP-infeasible, the other feasible-set members are tried in ascending score order.

Fallback. If $\mathcal{F} = \varnothing$ under the normal window, the scoring is repeated with $w_b = w_f = n$ (every position). If it is still empty, construction fails and returns `(None, inf)`. This never happened on airland1–13.

3.7 Pool generation and uniqueness

`etri_pool(inst, m, k)` returns up to k distinct schedules sorted by exact cost.

- The deterministic schedule goes in first.
- Randomised runs then use seeds `seed, seed+1, …`, up to `max_tries` $= 4k$ runs.
- Schedules are compared in **canonical form**: runways are identical, so the runway tuples are sorted lexicographically.
- After any batch that produced a duplicate, the temperature is raised: $\tau \leftarrow \min(1.5\,\tau,\ 10)$. This trades some quality for diversity on instances where most selection steps have one clear winner.

3.8 Parameters

Parameter	Default	Meaning
`candidates` K	4	candidate-set size (must be $\le H$)
`horizon` H	8	aircraft placed by each pilot run
`slots` T	4	cheapest slots per candidate scored by the pilot

Parameter	Default	Meaning
`w_back` w_b	6	window positions before the target-order slot
`w_fwd` w_f	1	window positions after the target-order slot
`temperature` τ	0.3	selection temperature in units of c_{unit} (randomised mode only)
`escalate`, `temperature_max`	1.5, 10	pool diversification (§3.7)

Measured sensitivity (deterministic, mean gap to BKS; single runway = airland8–13, multi-runway = airland9–13 with m = 2–4):

(K, H, T)	**Single runway**	**Multi-runway**
(3, 8, 3)	6.32%	0.03%
(4, 8, 4) (default)	**2.44%**	**0.03%**
(4, 8, 8)	2.15%	—
(6, 8, 4)	2.07%	—
(4, 10, 6)	2.12%	—
(3, 6, 8)	4.59%	—
(4, 12, 8)	2.27%	0.60%
(1, 8, 8)	2.72%	5.12%

The default is the best setting that holds up on both runway regimes. Larger settings help some single-runway jobs but not consistently, and they cost more time. Widening w_b from 6 to 12 changed nothing. For the temperature, on the single-runway jobs (6 seeds each), $\tau = 0.3$ gave a pool best of 1.95%, a pool mean of 2.65% and 81% unique schedules; $\tau = 1.0$ gave 2.68%, 5.39% and 92%.

3.9 Pseudocode

```
ETRI(inst, m, rng=None, prm):
    Π^pend ← stable argsort of δ;   π_ρ ← (), F_ρ ← 0 for every runway ρ
    while Π^pend ≠ ∅:
        𝓕 ← SCORE(w_b, w_f)
        if 𝓕 = ∅: 𝓕 ← SCORE(n, n)                      # full-range fallback
        if 𝓕 = ∅: return FAIL
        if rng = None: order ← 𝓕 by (score, candidate order)
        else:          j0 ~ softmax(−(score − min)/(τ·c_unit))
                       order ← [j0] + remaining 𝓕 by score
        for j in order:                                  # commit, §3.6
            for (Φ, ρ, p) in scored slots of j, ascending:
                c ← EXACT-COST(insert(π_ρ, j, p))        # PAVA, or LP if flagged
                if c < ∞: π_ρ ← insert(π_ρ, j, p); F_ρ ← c; remove j from Π^pend
                          go to next step of the outer loop
        return FAIL
    return Π, Σ_ρ F_ρ

SCORE(wb, wf):
    Λ ← Π^pend[0..H-1];   𝓕 ← []
    for j in Π^pend[0..K-1]:
        O ← finite slots of j in window(wb, wf), stably sorted by Δ = f_c − F_ρ
        S ← []
        for (ρ, p) in O[0..T-1]:
            Φ ← PILOT(j, ρ, p, Λ)
            if Φ < ∞: S.append((Φ, ρ, p))
        if S ≠ []: 𝓕.append((min S, j, sorted S))
    return 𝓕
```

```
PILOT(j, ρ, p, Λ):                                    # works on a copy
    insert j at (ρ, p)
    for k in Λ \ {j}, in order:
        O ← finite slots of k;   if O = ∅: return ∞
        insert k at O[0]
    return total cost of the copy
```

3.10 Deterministic rules (reproduction recipe)

Following these rules with `rng=None` and default parameters reproduces `etri(inst, m)` exactly. Tolerances are as in the code: 10^{-9} for windows, 10^{-12} for PAVA merges and 10^{-7} for pairwise checks.

- **E1.** Sort aircraft by δ with a stable sort (equal targets by index): this is the pending list. Start with m empty runways of cost 0.
- **E2.** Take the first $K = 4$ pending aircraft as candidates and the first $H = 8$ as the look-ahead list Λ.
- **E3.** For each candidate j and each runway ρ, compute $q_\rho(j)$ (strict $>$, §3.3). For every position p in $[\max(0, q-6), \min(L, q+1)]$, insert j and compute the PAVA value f_c, ignoring the exactness flag. Keep the finite values and sort them stably by $f_c - F_\rho$ (enumeration order: ρ ascending, then p ascending).
- **E4.** For each of the first $T = 4$ options, run the pilot (§3.4): insert j there, then insert the other members of Λ in list order, each at its own first option from E3 computed on the current copy. If some aircraft has no finite option, that pilot is ∞. Otherwise record (Φ, ρ, p) with Φ the copy's total cost.
- **E5.** A candidate's score is its smallest Φ, and its scored slots are sorted by (Φ, ρ, p). Candidates without a finite Φ drop out. If none remain, redo E3–E5 with the window $[0, L]$ on every runway; if still none remain, stop with failure.
- **E6.** Choose the candidate with the smallest score; ties go to the earliest in the candidate list.
- **E7.** For that candidate's slots in order, compute the exact runway cost of the new sequence: PAVA, and if the pairwise check flags it, the pruned LP of §3.2. Commit the first slot with a finite cost: insert the aircraft, set the runway cost, and remove it from the pending list. If no slot works, try the next candidate in score order (ties by candidate order).
- **E8.** Repeat E2–E7 until nothing is pending. The output is the runway sequences and $\sum_\rho F_\rho$, which equals the Stage-2 LP objective.

Randomised variant: replace E6 with the softmax draw of §3.5. Every other rule is unchanged.

3.11 Worked example: airland1, deterministic

m = 1 (generated from the code and asserted equal to `etri()`; data in Appendix A):

Step	Candidates	Pilot scores	Picked	Pos	Sequence after	Cost
1	2,3,4,5	620, 620, 620, 620	2	0	[2]	0
2	3,4,5,6	700 ×4	3	1	[2,3]	0
3	4,5,6,7	700 ×4	4	2	[2,3,4]	0
4	5,6,7,8	700 ×4	5	3	[2,3,4,5]	0
5	6,7,8,0	700 ×4	**6**	**3**	[2,3,4,**6**,5]	330

Step	Candidates	Pilot scores	Picked	Pos	Sequence after	Cost
6	7,8,0,9	700 ×4	7	5	[2,3,4,6,5,7]	540
7	8,0,9,1	700 ×4	8	6	[2,3,4,6,5,7,8]	570
8	0,9,1	700 ×3	0	7	[2,3,4,6,5,7,8,0]	680
9	9,1	700 ×2	9	8	[2,3,4,6,5,7,8,0,9]	700
10	1	700	1	9	[2,3,4,6,5,7,8,0,9,1]	**700**

Reading the table:

- **Steps 1–4.** The scores tie, so E6 takes the earliest candidate.
- **Step 5.** All candidates score 700 again, so aircraft 6 is picked by candidate order. Its cheapest slot is position 4, with an immediate increase of +150 against +330 at position 3. The pilot scores both slots at exactly 700, though: placing 6 *ahead of* 5 costs more now but nothing over the next 8 insertions. The (Φ, ρ, p) tie rule then picks the lower position, 3. This is a deviation from target order that a myopic rule would not make.
- **Steps 2–10.** The score (700) already equals the final cost, which is the optimum for airland1 with one runway. The pilot horizon of 8 covers almost the whole instance, so it sees the final cost early.

m = 2: final schedule [2,3,4,5,7,8,9,1] · [6,0] with cost **90.00** (optimal). Every score ties at 90, so the candidate-order tie rule decides every step. Aircraft 6 opens runway 2 at step 5 and aircraft 0 joins it at step 8. Appendix B.2 has the full trace.

3.12 Complexity derivation

Notation: k is the number of aircraft scheduled, $\bar{L} = k/m$ the mean runway length, $W = w_b + w_f + 1$, and $C(L)$ the cost of one PAVA call on length L.

PAVA, $C(L)$.

- Prefix sums: $O(L)$.
- Every position is pushed once and every merge removes a block, so there are at most $L - 1$ merges. Merging blocks of sizes a and b (sorted merge plus minimiser scan) costs $O(a + b)$.
  - Worst case (each new position merges into one growing block): $\sum_{k=1}^{L} O(k) = O(L^2)$.
  - Blocks of bounded size β: $O(\beta L)$.
- Time recovery and cost: $O(L)$. Pairwise check: $O(\kappa L)$ (§3.2).

$$C(L) = O\left(L(\beta + \kappa)\right) \text{ typical}, \qquad C(L) = O(L^2) \text{ worst}.$$

Option list of one aircraft. Per runway: q_ρ in $O(L_\rho)$, then W slots each costing an $O(L_\rho)$ insertion copy plus $C(L_\rho + 1)$. Adding a sort of $O(mW\log(mW))$ gives

$$O_{slots} = O\left(m\,W\,C(\bar{L})\right).$$

One pilot run. Copying the state costs $O(mn)$ (the full $m \times n$ buffer). Then come $H - 1$ option lists and commits:

$$O_{pilot} = O\left(mn + H\,m\,W\,C(\bar{L})\right).$$

Runway lengths grow by at most H during a pilot, which is absorbed because $\bar{L} + H \le n$.

One construction step. K candidates, each with one option list and T pilots, plus an $O(C(L))$ commit (rarely an LP) and $O(n)$ list maintenance:

$$T_{step}(k) = O\left(KT(mn + H\,m\,W\,C(k/m)) \right).$$

Whole construction (n steps).

- Typical, $C(L) = cL$: $\sum_{k<n} KT(mn + HWk) = KT(mn^2 + HW\frac{n(n-1)}{2})$.

$$T_{ETRI} = O\left(KT(m + HW)\,n^2 \right).$$

- Worst case, $C(L) = L^2$: $\sum_k KTHmW(k/m)^2 = KTHW\frac{(n-1)n(2n-1)}{6m}$, so $O\left(KTHW\,n^3/m + KTm\,n^2 \right)$.

With the defaults $KTHW = 1024$ and $m \le 5 \ll HW = 64$, the typical cost is about $1024 \cdot n^2/2$ PAVA position-operations.

Fallback steps (window $[0, L]$): the window size W is replaced by $L + 1$ for that step only. This is rare.

Pool. At most $4k + 1$ constructions, run in parallel across `n_workers` processes.

Memory. $O(n^2)$ for s, $O(mn)$ for the state, $O(mn)$ per live pilot copy, and memoised exact costs (one float per distinct committed runway sequence).

Empirical check (one deterministic construction, best of 3 runs; plotted in Figure 1, §2.11).

n	100	150	200	250	500
m = 1 (s)	0.097	0.198	0.310	0.516	1.804
m = 2 (s)	0.086	0.155	0.239	0.348	1.111

The fitted exponent between $n = 100$ and 500 is 1.8 for one runway and 1.6 for two, consistent with the typical $O(n^2)$ bound plus sub-quadratic per-call overhead. At $n = 500$, $m = 1$, ETRI needs 1.8 s and TC-RBI 7.4 s.

3.13 Properties and limitations

- **Feasible and exactly costed.** Every committed runway is priced with the exact Stage-2 optimum, and a slot the LP rejects is never committed. Every schedule produced in the benchmark (the deterministic one plus up to 8 pool members for each of the 36 jobs) passed `alp_core.audit` (§5).
- **Pilot values are lower bounds inside the pilot.** The pilot uses PAVA values without the exactness flag. On triangle-violating instances such as airland8, a pilot can therefore be optimistic, but the commit step is always exact.
- **Single runway, congested instances.** On airland10–13 with one runway, construction alone stays 1.3–4.6% above BKS. Near-optimal orders differ from target order by a few swaps inside rushes, which an 8-aircraft greedy pilot cannot always foresee. SA closes most of that gap.

- **Limited diversity on tiny or tight instances.** airland6 with one runway has essentially one feasible sequence (its best schedule has no feasible adjacent swap), so the pool returns 1 schedule. airland7 (m = 1) also returns 1, and airland1 and airland3 (m = 1) return 2.
- **Escalation trades quality for diversity.** On airland13 with m = 4 the deterministic run is optimal (89.95). Once duplicates appear the temperature rises, and the pool mean climbs to 107.4.
- **No improvement phase,** by design; ETRI is a construction heuristic for seeding SA.

---

4. Side-by-side comparison

Aspect	TC-RBI	ETRI
Which aircraft compete	the conflict queue: adaptive, at least $m+1$, at most 14	the first $K=4$ pending aircraft by target time
Look-ahead	resolve the whole queue	greedy completion of the next $H=8$
Slots scored per aircraft	every slot in the window ($\leq 8m$)	the $T=4$ cheapest
Selection score	queue-resolution cost (generalised pairwise regret)	pilot cost
Insertion cost and feasibility	exact (PAVA, LP when flagged)	exact (PAVA, LP when flagged)
Weights or tuning	none	none
Randomness	softmax over queue members	softmax over candidates
Time per construction, $m=1$, $n=500$	7.4 s	1.8 s
Deterministic mean gap, all 36 jobs (single / multi-runway)	0.47% / 0.05%	1.15% / 0.01%
Pool-of-8 best, mean gap (single / multi-runway)	0.34% / 0.00%	0.95% / 0.01%
Shared code	`alp_core.py`: PAVA, slots, exact LP, pools, benchmark driver	same

5. Validation procedure

These checks were applied to every result reported here:

1. **PAVA vs HiGHS:** 780 sequences (random perturbations of target order, 1–3 runway splits, all 13 instances). There were 0 mismatches among the 530 that PAVA flagged as exact; 227 were infeasible for both methods, and 23 were flagged as not exact (LP needed).
2. **Schedule audit** (`alp_core.audit`):
   - **A1** every aircraft appears exactly once;
   - **A2** feasibility by earliest-time propagation over all predecessor pairs, in plain Python and independent of PAVA;
   - **A3** every window and every ordered-pair separation holds at the optimal landing times, and the cost recomputed from those times matches the reported cost.

   Every schedule of both heuristics passed: the deterministic one plus up to 8 pool members per job, for all 36 jobs. The audit was itself checked against an independent

full-pairwise LP implementation, giving identical costs and verdicts. It also rejects a schedule with a missing aircraft, an infeasible order, or a misreported cost.

3. **Independent single-runway audit:** the best single-runway values found during development (12292.20 for airland10 and 37165.39 for airland13) also passed an independent nine-group constraint audit (C1–C9).
4. **Trace reproduction:** the traces in §2.9 and §3.11 and in Appendix B were generated by instrumented copies of the main loops and asserted equal to `tc_rbi()` and `etri()`.
5. **Refactor check:** after moving the shared kernels to `alp_core.py`, ETRI's deterministic runs, randomised runs and pools were compared with the pre-refactor code on seven jobs, including airland8 (the LP fallback path). All were identical.

6. Results on airland1–13

Gaps are relative to the reference values in `alp_core.REFERENCE`. All schedules are LP-feasible and pass the pairwise audit.

6.1 Deterministic construction

Job	n	BKS	TC-RBI	gap	ETRI	gap
airland1 m1	10	700	700.00	0.00%	700.00	0.00%
airland1 m2	10	90	90.00	0.00%	90.00	0.00%
airland2 m1	15	1480	1480.00	0.00%	1480.00	0.00%
airland2 m2	15	210	210.00	0.00%	210.00	0.00%
airland3 m1	20	820	820.00	0.00%	820.00	0.00%
airland3 m2	20	60	60.00	0.00%	60.00	0.00%
airland4 m1	20	2520	2520.00	0.00%	2540.00	+0.79%
airland4 m2	20	640	640.00	0.00%	640.00	0.00%
airland4 m3	20	130	130.00	0.00%	130.00	0.00%
airland5 m1	20	3100	3100.00	0.00%	3100.00	0.00%
airland5 m2	20	650	650.00	0.00%	650.00	0.00%
airland5 m3	20	170	170.00	0.00%	170.00	0.00%
airland6 m1	30	24442	24442.00	0.00%	24442.00	0.00%
airland6 m2	30	554	554.00	0.00%	554.00	0.00%
airland7 m1	44	1550	1550.00	0.00%	1550.00	0.00%
airland8 m1	50	1950	1950.00	0.00%	1950.00	0.00%
airland8 m2	50	135	135.00	0.00%	135.00	0.00%
airland9 m1	100	5611.7	5711.28	+1.77%	5635.23	+0.42%
airland9 m2	100	444.1	444.10	0.00%	444.56	+0.10%
airland9 m3	100	75.75	75.75	0.00%	75.75	0.00%
airland10 m1	150	12310.7	12400.92	+0.73%	12870.81	+4.55%
airland10 m2	150	1143.7	1143.70	0.00%	1143.70	0.00%
airland10 m3	150	205.21	205.21	0.00%	205.21	0.00%
airland10 m4	150	34.22	34.22	0.00%	34.22	0.00%
airland11 m1	200	12418.3	12529.98	+0.90%	12838.82	+3.39%
airland11 m2	200	1330.91	1330.91	0.00%	1330.91	0.00%
airland11 m3	200	253.07	253.07	0.00%	253.07	0.00%

Job	n	BKS	TC-RBI	gap	ETRI	gap
airland11 m4	200	54.53	54.53	0.00%	54.53	0.00%
airland12 m1	250	16152.7	16452.51	+1.86%	16854.33	+4.34%
airland12 m2	250	1695.62	1714.50	+1.11%	1697.34	+0.10%
airland12 m3	250	221.97	221.97	0.00%	221.97	0.00%
airland12 m4	250	2.44	2.44	0.00%	2.44	0.00%
airland13 m1	500	37268.1	37589.42	+0.86%	37817.98	+1.48%
airland13 m2	500	3920.39	3920.39	0.00%	3922.40	+0.05%
airland13 m3	500	673.85	673.85	0.00%	673.85	0.00%
airland13 m4	500	89.95	89.95	0.00%	89.95	0.00%

6.2 Pools of 8 distinct schedules

Each pool is the deterministic schedule plus randomised runs (seed 0, $\tau = 0.3$, escalation on duplicates). “uniq” is the number of distinct schedules returned (at most 8). Times are for the whole pool.

Job	TC-RBI pool best	mean	uniq	ETRI pool best	mean	uniq	TC-RBI pool time	ETRI pool time
airland1 m1	0.00%	0.00%	2	0.00%	0.00%	2	0.1s	0.1s
airland1 m2	0.00%	0.00%	4	0.00%	0.00%	4	0.1s	0.1s
airland2 m1	0.00%	+0.51%	8	0.00%	+0.77%	7	0.1s	0.2s
airland2 m2	0.00%	0.00%	8	0.00%	0.00%	8	0.0s	0.1s
airland3 m1	0.00%	0.00%	2	0.00%	0.00%	2	0.7s	0.2s
airland3 m2	0.00%	0.00%	8	0.00%	0.00%	8	0.1s	0.1s
airland4 m1	0.00%	0.00%	8	0.00%	+0.40%	8	0.2s	0.1s
airland4 m2	0.00%	+0.78%	8	0.00%	0.00%	8	0.1s	0.1s
airland4 m3	0.00%	0.00%	8	0.00%	0.00%	8	0.2s	0.1s
airland5 m1	0.00%	0.00%	8	0.00%	0.00%	8	0.3s	0.1s
airland5 m2	0.00%	+1.54%	8	0.00%	+1.15%	8	0.1s	0.1s
airland5 m3	0.00%	+2.21%	8	0.00%	+4.41%	8	0.2s	0.1s
airland6 m1	0.00%	0.00%	1	0.00%	0.00%	1	0.7s	0.2s
airland6 m2	0.00%	+0.65%	8	0.00%	0.00%	8	0.1s	0.1s
airland7 m1	0.00%	0.00%	1	0.00%	0.00%	1	2.1s	0.6s
airland8 m1	0.00%	+0.77%	4	0.00%	+1.54%	8	5.4s	2.3s

Job	TC-RBI pool best	mean	uniq	ETRI pool best	mean	uniq	TC-RBI pool time	ETRI pool time
airland8 m2	0.00%	+1.85%	8	0.00%	+1.85%	8	0.4s	0.5s
airland9 m1	+0.01%	+1.88%	8	+0.01%	+1.75%	8	7.5s	1.4s
airland9 m2	0.00%	+0.94%	8	+0.10%	+1.37%	8	1.4s	0.9s
airland9 m3	0.00%	0.00%	8	0.00%	0.00%	8	2.2s	0.9s
airland10 m1	+0.73%	+9.10%	8	+4.55%	+12.39%	8	16.6s	2.0s
airland10 m2	0.00%	+1.37%	8	0.00%	+1.22%	8	3.5s	1.8s
airland10 m3	0.00%	+4.05%	8	0.00%	+0.05%	8	4.2s	2.0s
airland10 m4	0.00%	0.00%	8	0.00%	0.00%	8	7.1s	1.9s
airland11 m1	+0.90%	+3.27%	8	+2.91%	+3.56%	8	24.1s	3.2s
airland11 m2	0.00%	+0.83%	8	0.00%	+1.27%	8	3.6s	2.5s
airland11 m3	0.00%	+0.49%	8	0.00%	+0.49%	8	6.0s	2.7s
airland11 m4	0.00%	+6.18%	8	0.00%	+3.74%	8	10.5s	3.3s
airland12 m1	+1.86%	+4.16%	8	+3.62%	+4.64%	8	40.8s	5.2s
airland12 m2	0.00%	+1.32%	8	+0.10%	+1.17%	8	6.4s	3.7s
airland12 m3	0.00%	+2.12%	8	0.00%	+1.34%	8	9.3s	3.8s
airland12 m4	0.00%	0.00%	8	0.00%	0.00%	8	15.3s	4.1s
airland13 m1	+0.86%	+3.70%	8	+1.28%	+2.27%	8	156.2s	19.0s
airland13 m2	0.00%	+1.27%	8	+0.05%	+0.86%	8	25.8s	11.4s
airland13 m3	0.00%	+3.92%	8	0.00%	+2.45%	8	32.9s	11.8s
airland13 m4	0.00%	+2.07%	8	0.00%	+19.43%	8	51.3s	12.1s

Note on references. The single-runway references for airland10 (12310.70) and airland13 (37268.12) are not optimal. Schedules with **12292.20** and **37165.39** were found during ETRI development and passed both repository audits (§5).

6.3 Randomness, diversity and local optimality

Both heuristics randomise only *which aircraft* is inserted next. The draw is a softmax over the feasible candidates (ETRI) or the conflict queue (TC-RBI), with temperature τ in units of c_{unit}.

- With $\tau = 0.3$, a candidate whose score is $0.3\,c_{unit}$ worse than the best is drawn with relative weight $e^{-1} = 0.37$, and one $1\,c_{unit}$ worse with weight 0.036.
- So near-ties, which are where alternative good schedules come from, are explored, and clearly worse choices are almost never made.

`etri_pool` and `tc_rbi_pool` collect distinct schedules (deduplicated in canonical form) and raise τ by a factor of 1.5 whenever a batch repeats a schedule.

Local optimality. A construction heuristic does not guarantee that its output is a local optimum. Guaranteeing that needs a descent phase, which is a search method and is deliberately left to SA. To quantify the point, every pool schedule was tested for an improving neighbour: within-runway relocation or swap of up to 8 positions, or a cross-runway transfer or swap near the target position, all under exact costs. A schedule with no improving neighbour is a local optimum.

Pool of 8, 14 hardest jobs, $\tau = 0.3$	**Best gap**	**Mean gap**	**Unique**	**Already local optima**
TC-RBI	**0.35%**	2.43%	94%	29%
ETRI	0.93%	2.51%	96%	28%

The 14 jobs are airland8–13 with $m = 1$ and airland9–13 with $m = 2$–3; gaps are relative to the best known values.

Points to note:

- About 29% of constructed schedules are already local optima. They are mostly multi-runway schedules and small single-runway instances.
- None of the airland10–13 single-runway schedules is a local optimum: congested single-runway sequences always admit some improving swap that a greedy construction cannot see.
- Both heuristics give near-optimal, mostly distinct starting points. SA's first iterations take them to local optima.

6.4 Effect on SA

This test used an exact-objective multi-runway SA (60 s, 4 workers, single runs) with ETRI seeds. The portfolio column uses a portfolio of nine dispatching and insertion seed heuristics:

Job	**Portfolio seeds → SA**	**ETRI pool (k = 4) → SA**
airland13 m2	4659.60 → 4659.60 (+18.86%)	3922.40 → 3922.40 (+0.05%)
airland13 m3	863.28 → 863.28 (+28.11%)	673.85 → 673.85 (0.00%)
airland10 m1	17749.52 → 12317.82 (+0.06%)	12870.81 → **12300.40 (−0.08%)**
airland12 m1	18414.09 → 16298.75 (+0.90%)	16854.33 → 16338.61 (+1.15%)

At $n = 500$ the SA made no progress in 60 s from either start, so the seed quality decides the result there. TC-RBI's deterministic schedule for airland13 with m = 2 is already at the reference value (3920.39).

7. Using the heuristics as SA seeds

```python
import numpy as np
from const_heuristics.alp_core import load_instance
from const_heuristics.ramp_rbi import tc_rbi, tc_rbi_pool, TCRBIParams
from const_heuristics.etri import etri, etri_pool, ETRIParams

inst = load_instance("data/airland12.txt")

# multi-runway search: a list of per-runway sequences per start
starts = [seqs for _, seqs in tc_rbi_pool(inst, m=2, k=8, seed=0)]
starts += [seqs for _, seqs in etri_pool(inst, m=2, k=8, seed=0)]

# single-runway search: one flat sequence per start
starts_1 = [seqs[0] for _, seqs in tc_rbi_pool(inst, m=1, k=8)]

# one deterministic / one randomised schedule
seqs, cost = tc_rbi(inst, 2)
seqs, cost = tc_rbi(inst, 2, rng=np.random.default_rng(7), params=TCRBIParams(temperature=0.5))
```

The two pools can be merged and deduplicated with `alp_core.canonical`.

8. Reproducibility notes

- **Environment.** Python 3.13 with NumPy, SciPy ≥ 1.9 (HiGHS through `linprog`) and Numba. Numba kernels compile on first call (about 2–4 s) and are cached in `__pycache__`.
- **Both heuristics** are fully determined in deterministic mode by their rule lists (§2.8, §3.10). Every sort that can see ties is stable (`kind="stable"` or `"mergesort"`, or Python's stable `sort`), and cost ties are exact floating-point equalities. Randomised runs are reproducible from the NumPy `Generator` seed.
- **Imports.** `from const_heuristics.ramp_rbi import tc_rbi` from the repository root, or run `python const_heuristics/ramp_rbi.py` (likewise for `etri`). Both forms resolve `alp_core`.
- **Benchmark drivers.** Set the `CONFIGURE HERE` block in `ramp_rbi.py` or `etri.py` (`JOBS`, `POOL_SIZE`, `SEED`, `N_WORKERS`) and run the file.

9. References

- Ahuja, R. K., & Orlin, J. B. (2001). A fast scaling algorithm for minimizing separable convex functions subject to chain constraints. *Operations Research*, 49(5), 784–789.
- Beasley, J. E., Krishnamoorthy, M., Sharaiha, Y. M., & Abramson, D. (2000). Scheduling aircraft landings — the static case. *Transportation Science*, 34(2), 180–197.
- Best, M. J., Chakravarti, N., & Ubhaya, V. A. (2000). Minimizing separable convex functions subject to simple chain constraints. *SIAM Journal on Optimization*, 10(3), 658–672.
- Duin, C., & Voß, S. (1999). The pilot method: A strategy for heuristic repetition with application to the Steiner problem in graphs. *Networks*, 34(3), 181–191.
- Feo, T. A., & Resende, M. G. C. (1995). Greedy randomized adaptive search procedures. *Journal of Global Optimization*, 6(2), 109–133.
- Potvin, J.-Y., & Rousseau, J.-M. (1993). A parallel route building algorithm for the vehicle routing and scheduling problem with time windows. *European Journal of Operational Research*, 66(3), 331–340.

- Salehipour, A. (2020). An algorithm for single- and multiple-runway aircraft landing problem. *Mathematics and Computers in Simulation*, 175, 179–191.

Appendix A — airland1 data

j	r_j	δ_j	d_j	g_j	h_j
0	129	155	559	10	10
1	195	258	744	10	10
2	89	98	510	30	30
3	96	106	521	30	30
4	110	123	555	30	30
5	120	135	576	30	30
6	124	138	577	30	30
7	126	140	573	30	30
8	135	150	591	30	30
9	160	180	657	30	30

Separation matrix s_{ij} (row i lands before column j):

```
     0   1   2   3   4   5   6   7   8   9
 0   0   3  15  15  15  15  15  15  15  15
 1   3   0  15  15  15  15  15  15  15  15
 2  15  15   0   8   8   8   8   8   8   8
 3  15  15   8   0   8   8   8   8   8   8
 4  15  15   8   8   0   8   8   8   8   8
 5  15  15   8   8   8   0   8   8   8   8
 6  15  15   8   8   8   8   0   8   8   8
 7  15  15   8   8   8   8   8   0   8   8
 8  15  15   8   8   8   8   8   8   0   8
 9  15  15   8   8   8   8   8   8   8   0
```

Derived: $\bar{s} = 10.3778$, $s_{\max} = 15$, $c_{unit} = 269.8222$.

Appendix B — full traces

B.1 TC-RBI, airland1, m = 2, deterministic

Each line gives the queue, then `member: D* @(runway, position)`.

```
step 1: Q=[2, 3, 4] | D* 2: 0.00 @(1,0); 3: 0.00 @(1,0); 4: 0.00 @(1,0)
        pick 2 -> runway 1 pos 0
        runways=[[2], []] cost=0.00
step 2: Q=[3, 4, 5, 6, 7] | D* 3: 90.00 @(1,1); 4: 90.00 @(1,1); 5: 90.00 @(1,1); 6: 90.00 @(1,1); 7:
150.00 @(1,1)
        pick 3 -> runway 1 pos 1
        runways=[[2, 3], []] cost=0.00
step 3: Q=[4, 5, 6, 7] | D* 4: 90.00 @(1,2); 5: 90.00 @(1,2); 6: 90.00 @(1,2); 7: 150.00 @(1,2)
        pick 4 -> runway 1 pos 2
        runways=[[2, 3, 4], []] cost=0.00
step 4: Q=[5, 6, 7] | D* 5: 90.00 @(1,3); 6: 90.00 @(1,3); 7: 150.00 @(1,3)
        pick 5 -> runway 1 pos 3
```

```
        runways=[[2, 3, 4, 5], []] cost=0.00
step 5: Q=[6, 7, 8, 0] | D* 6: 90.00 @(2,0); 7: 90.00 @(1,4); 8: 90.00 @(1,4); 0: 90.00 @(1,4)
        pick 6 -> runway 2 pos 0
        runways=[[2, 3, 4, 5], [6]] cost=0.00
step 6: Q=[7, 8, 0] | D* 7: 90.00 @(1,4); 8: 90.00 @(1,4); 0: 90.00 @(1,4)
        pick 7 -> runway 1 pos 4
        runways=[[2, 3, 4, 5, 7], [6]] cost=90.00
step 7: Q=[8, 0, 9] | D* 8: 90.00 @(1,5); 0: 90.00 @(1,5); 9: 90.00 @(1,5)
        pick 8 -> runway 1 pos 5
        runways=[[2, 3, 4, 5, 7, 8], [6]] cost=90.00
step 8: Q=[0, 9, 1] | D* 0: 90.00 @(2,1); 9: 90.00 @(1,6); 1: 90.00 @(1,6)
        pick 0 -> runway 2 pos 1
        runways=[[2, 3, 4, 5, 7, 8], [6, 0]] cost=90.00
step 9: Q=[9, 1] | D* 9: 90.00 @(1,6); 1: 90.00 @(1,6)
        pick 9 -> runway 1 pos 6
        runways=[[2, 3, 4, 5, 7, 8, 9], [6, 0]] cost=90.00
step 10: Q=[1] | D* 1: 90.00 @(1,7)
        pick 1 -> runway 1 pos 7
        runways=[[2, 3, 4, 5, 7, 8, 9, 1], [6, 0]] cost=90.00
matches tc_rbi(): ([[2, 3, 4, 5, 7, 8, 9, 1], [6, 0]], 90.0)
```

B.2 ETRI, airland1, m = 2, deterministic

Each line gives `candidate: pilot score @(runway, position)`.

```
step 1: C=[2, 3, 4, 5] | scores 2: 90.00 @(1,0); 3: 90.00 @(1,0); 4: 90.00 @(1,0); 5: 90.00 @(1,0) |
pick 2 -> runway 1 pos 0
        runways=[[2], []] cost=0.00
step 2: C=[3, 4, 5, 6] | scores 3: 90.00 @(1,1); 4: 90.00 @(1,1); 5: 90.00 @(1,1); 6: 90.00 @(1,1) |
pick 3 -> runway 1 pos 1
        runways=[[2, 3], []] cost=0.00
step 3: C=[4, 5, 6, 7] | scores 4: 90.00 @(1,2); 5: 90.00 @(1,2); 6: 90.00 @(1,2); 7: 150.00 @(1,2) |
pick 4 -> runway 1 pos 2
        runways=[[2, 3, 4], []] cost=0.00
step 4: C=[5, 6, 7, 8] | scores 5: 90.00 @(1,3); 6: 90.00 @(1,3); 7: 150.00 @(1,3); 8: 90.00 @(1,3) |
pick 5 -> runway 1 pos 3
        runways=[[2, 3, 4, 5], []] cost=0.00
step 5: C=[6, 7, 8, 0] | scores 6: 90.00 @(2,0); 7: 90.00 @(1,4); 8: 90.00 @(1,4); 0: 90.00 @(1,4) |
pick 6 -> runway 2 pos 0
        runways=[[2, 3, 4, 5], [6]] cost=0.00
step 6: C=[7, 8, 0, 9] | scores 7: 90.00 @(1,4); 8: 90.00 @(1,4); 0: 90.00 @(1,4); 9: 90.00 @(1,4) |
pick 7 -> runway 1 pos 4
        runways=[[2, 3, 4, 5, 7], [6]] cost=90.00
step 7: C=[8, 0, 9, 1] | scores 8: 90.00 @(1,5); 0: 90.00 @(1,5); 9: 90.00 @(1,5); 1: 90.00 @(1,5) |
pick 8 -> runway 1 pos 5
        runways=[[2, 3, 4, 5, 7, 8], [6]] cost=90.00
step 8: C=[0, 9, 1] | scores 0: 90.00 @(2,1); 9: 90.00 @(1,6); 1: 90.00 @(1,6) | pick 0 -> runway 2 pos
1
        runways=[[2, 3, 4, 5, 7, 8], [6, 0]] cost=90.00
step 9: C=[9, 1] | scores 9: 90.00 @(1,6); 1: 90.00 @(1,6) | pick 9 -> runway 1 pos 6
        runways=[[2, 3, 4, 5, 7, 8, 9], [6, 0]] cost=90.00
step 10: C=[1] | scores 1: 90.00 @(1,7) | pick 1 -> runway 1 pos 7
        runways=[[2, 3, 4, 5, 7, 8, 9, 1], [6, 0]] cost=90.00
matches etri(): ([[2, 3, 4, 5, 7, 8, 9, 1], [6, 0]], 90.0)
```